Concept Graph Attention: Sequence Modeling over Fixed Concept Memory

Anonymous Author(s)
Affiliation
Address
email

Abstract

Self-attention stores one key-value entry per previous token and retrieves from that token table. This gives expressive content-addressed recall, but it ties the memory basis to token events and makes the read state grow with context length. We propose *Concept Graph Attention* (CGA), a causal sequence layer that keeps the continuous readout of attention but replaces token memory with a fixed bank of runtime concepts. Each concept slot stores an address, value memory, count statistics, and trainable retention dynamics. Tokens read from edge-eligible concept slots with attention-style softmax weights, then write back by gated sufficient-statistic updates. CGA therefore changes where attention is computed: from token positions to a compact concept bank whose contents evolve over the prefix.

Theoretically, fixed-bank CGA is an attention-compatible state layer with a clear scan-friendly update: reads are differentiable concept attention, while writes are local associative-memory repairs with detached routing and differentiable value/gate paths. This gives CGA *test-time concept learning*: at inference, model weights remain fixed, but the concept memory is learned from the observed prefix by a forward-only update. This separates slow neural training from fast runtime state mutation without requiring an inference-time backward pass. Empirically, we evaluate CGA in the PoST/Zoology sequence-modeling framework on state-equalized multi-query associative recall and a small causal language-model setup. The intended comparison class is modern efficient attention alternatives such as Mamba, RWKV, GDN, and TTT-style stateful layers: CGA keeps attention-like retrieval, but moves the persistent state from token memory or learned test-time weights to a bounded, updateable concept memory.

1 Introduction

Attention can be read as a memory operation. Given a prefix, a Transformer stores one key-value entry per token and later queries retrieve a weighted combination of those token values. This token table is powerful because the read is continuous and differentiable, but it fixes the unit of memory: the model attends to token events. Long-context architectures have explored recurrent states, state-space models, linear memories, sparse attention, and inference-time optimizers. These approaches change the systems tradeoff, but the central memory question remains: can the model attend over a smaller set of learned runtime concepts instead of attending over every previous token?

We propose *Concept Graph Attention* (CGA). CGA keeps the familiar attention read,

$$\mathrm{read}(q_t) = \sum_{j=1}^{C} \mathrm{softmax}_j(s(q_t, c_{tj}))\, m_{tj},$$

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

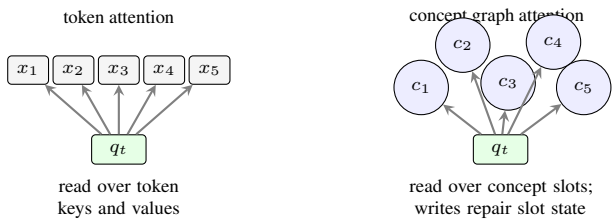

Figure 1: CGA changes the memory object of attention. Standard attention reads from token memory. CGA performs a continuous attention read over a fixed bank of runtime concept slots whose contents are updated causally.

but changes the objects being read. Instead of a token-indexed key-value table, each layer maintains a fixed bank of C concept slots. Each slot has an address c_{tj}, value memory m_{tj}, and count statistics that evolve causally over the prefix. The read is attention over concepts; the write is a gated sufficient-statistic update into the concept bank. Thus the layer is attention-like in its readout but recurrent-memory-like in its state:

$$\begin{aligned} \text{Transformer: tokens} &\rightarrow \text{attention over token memory,} \\ \text{CGA: tokens} &\rightarrow \text{fixed concept memory} \rightarrow \text{attention over concepts.} \end{aligned}$$

The fixed-bank design is deliberate. In language modeling and long synthetic recall, the data stream is effectively unbounded; a useful memory layer should eventually use its available capacity. CGA makes the capacity explicit from the first step and focuses the model on the harder problem: which concepts should be read and how should each token update them? The number of memory slots is a concept budget C rather than the context length T, giving a dense read cost of $O(TC)$ when $C \ll T$. At a fixed runtime-state budget, CGA can trade the number of concept slots against the per-slot value width.

This gives a different answer to the same question that motivates test-time-training sequence layers [Sun et al., 2025]. TTT layers make the hidden state a small model and update that model by a self-supervised learning step on the test sequence. CGA instead makes the hidden state a concept-indexed memory and updates that memory by a forward-only sufficient-statistic rule. Both mechanisms let a model adapt to information observed at inference time. The difference is the object being learned:

$$\begin{aligned} \text{TTT:} &\quad \text{test-time learning of hidden model weights,} \\ \text{CGA:} &\quad \text{test-time learning of concept-memory contents.} \end{aligned}$$

We use "learning" here in the operational stateful sense: future predictions condition on a state estimated from the observed prefix. CGA does not update the global model parameters during inference and does not run an inference-time backward pass.

This paper makes three contributions.

- **Architecture.** We introduce fixed-bank Concept Graph Attention, a causal mixer with test-time concept learning: it reads continuously from concept slots and updates them with gated writes and trainable retention decay while model weights stay fixed at inference time.
- **Theory.** We formalize CGA as attention over an online sufficient-statistic memory, specifying the state space, causal read/write equations, state-size scaling, scan-friendly update structure, and backpropagation boundary needed to compare it to recurrent, attention-like, and TTT-style sequence mixers.
- **Evaluation.** We implement CGA in the PoST/Zoology framework and evaluate it as an attention-like sequence mixer on state-equalized multi-query associative recall, with small language-model experiments as the scaling target.

The claim boundary is important. CGA is not presented as a fully optimized replacement for every attention kernel. It is a method proposal for changing the memory basis of attention from tokens to runtime concepts. The current implementation trains slow projections by backpropagation, while the concept-memory update itself is a forward-time operation. This places CGA between attention, external memory, and efficient recurrent mixers: it uses differentiable content-addressed reads like

attention, but the state being read is a compact, updateable concept memory rather than the entire prefix token table or a set of test-time-trained hidden weights.

2 Concept Graph Attention

CGA is an attention layer whose memory is a fixed bank of runtime concepts rather than a table of token keys and values. For each sequence and layer, the state before block b is

$$\mathcal{M}_b = \left\{(a_j, s^k_{jh}, s^v_{jh}, n_{jh})_{h=1}^{H}\right\}_{j=1}^{C}.$$

Here a_j is a fixed slot prior used to break symmetry, s^k_{jh} is the key sufficient statistic for slot j and head h, s^v_{jh} is the value sufficient statistic, and n_{jh} is a count or mass statistic. All C concept slots are available from the beginning of the sequence. Each slot also has a fixed edge predicate $e_j = (u_j, b_j, \sigma_j)$ derived from the random slot priors. For the main implementation, u_j is the separating direction between slot j's prior and its nearest neighboring prior, b_j is the midpoint bias, and σ_j is the side containing slot j. The slot addresses and memories used by the read are

$$c_{jh} = \mathrm{normalize}\big(s^k_{jh}/\max(n_{jh}, 1)\big), \qquad m_{jh} = s^v_{jh}/\max(n_{jh}, 1).$$

At initialization, $n_{jh} = n_0$, $s^k_{jh} = n_0 a_{jh}$, and $s^v_{jh} = 0$. The concept budget C is a model hyperparameter analogous to a recurrent-state budget. The value memory may use a concept bottleneck dimension d^c_v smaller than the ordinary attention head value dimension. For address dimension d_a, value bottleneck d^c_v, and H heads, the persistent per-sequence state scales as

$$S_{\mathrm{CGA}} = C\,H(d_a + d^c_v + 1).$$

The formula counts per-head key sums, value sums, and counts. Slot priors and edge predicates are fixed layer buffers rather than per-sequence runtime state. The important scaling point is that S_{CGA} depends on C, not on the context length.

What learns at inference time. During training, CGA learns slow parameters: projections, output maps, write gates, and retention parameters. During inference, these slow parameters are fixed. What changes is the per-sequence concept memory $\mathcal{M}_b$. After each block, the layer updates key statistics, value statistics, and count statistics using the prefix it has just observed. We call this *test-time concept learning*: it is learning because later predictions condition on a concept state estimated online from the test prefix, but it is not test-time gradient descent on model weights. This distinction is central to the method. The hidden state is not a token cache, because writes aggregate information into concept slots; it is not a learned inner-loop model, because the update is a forward-only sufficient-statistic rule.

Dense concept read. Given normalized query q_{th} and concept addresses c_{jh}, CGA reads by continuous attention over edge-eligible concept slots. Let

$$\bar{q}_t = \mathrm{normalize}\left(\frac{1}{H}\sum_{h=1}^{H} q_{th}\right), \qquad E_t = \{j : \mathbf{1}[u_j^\top \bar{q}_t > b_j] = \sigma_j\}.$$

If E_t is empty, CGA falls back to all concept slots. The read is

$$\alpha_{thj} = \frac{\exp(\gamma q_{th}^\top c_{jh})}{\sum_{r\in E_t}\exp(\gamma q_{th}^\top c_{rh})}, \qquad z_{th} = \sum_{j\in E_t}\alpha_{thj} m_{jh}.$$

The scale γ is learned and initialized at the standard dot-product attention scale. The output projection maps the concatenated head readout $(z_{t1}, \ldots, z_{tH})$ back to d_{model}. Thus the read path is ordinary differentiable attention, but the keys and values are concept memories rather than token memories.

Gated concept write. CGA uses a read-before-write causal protocol. After producing z_t, the layer forms a write key, value, and gate:

$$k_t = Kx_t, \qquad v_t = Vx_t, \qquad g_t = \sigma(w_g^\top x_t + b_g).$$

For MQAR we use a one-token delayed write key so that a key token can bind to the following value token:

$$k_t^{\text{write}} = k_{t-1}, \qquad v_t^{\text{write}} = v_t.$$

For general language modeling, the delay can be set to zero or treated as a task-dependent write convention. The write is routed to the most compatible concept slot:

$$j_t = \arg\max_{j \le C} \frac{1}{H} \sum_{h=1}^{H} (k_{th}^{\text{write}})^\top c_{jh}$$

The route j_t is detached fast state. The value write and gate remain differentiable through future concept reads.

Trainable retention. CGA maintains a trainable retention decay over old concept statistics. For head h,

$$\lambda_{th} = \exp(A_h)\ \big(\delta_{\min} + (\delta_{\max} - \delta_{\min})\sigma(u_h^\top x_t + b_h)\big)\, r_{th}, \qquad \rho_{bh} = \exp\left(-\sum_{t \in b} \lambda_{th}\right),$$

where $r_{th} = t^{-\alpha_h}$ when position-adaptive scaling is enabled and $r_{th} = 1$ otherwise. The default parameterization of A_h is PoST-style:

$$A_h = A_0 + \sum_{i<h} \text{softplus}(\Delta_i),$$

which gives ordered decay spectra across heads. The implementation also exposes a free per-head A_h parameterization for ablation. This decay plays the same role as the A or retention gate in Mamba-style state-space layers, but it acts on concept sufficient statistics rather than on a fixed token-indexed recurrent state.

Block update. Within a block, CGA accumulates writes with scatter-add operations and commits them at the block boundary:

$$\begin{aligned} n_{j_t h} &\leftarrow \text{sg}(\rho_{bh}) n_{j_t h} + \text{sg}(g_t), \\ s_{j_t h}^k &\leftarrow \text{sg}(\rho_{bh}) s_{j_t h}^k + \text{sg}(g_t) k_{th}^{\text{write}}, \\ s_{j_t h}^v &\leftarrow \rho_{bh} s_{j_t h}^v + g_t v_{th}^{\text{write}}. \end{aligned}$$

Here sg denotes stop-gradient. The key and count updates are detached routing state. The value payload, write gate, and retention decay receive gradients through future reads. Because writes are sums over assigned observations and decay is multiplicative per block, the update is scan-friendly: implementations can fuse dense concept reads and use parallel scatter reductions for the write side.

Relation to ordinary attention. Let $K_{\text{tok}}, V_{\text{tok}}$ be token-level keys and values. Standard attention computes

$$\text{Attn}(X)_t = \sum_{s<t} \text{softmax}_s(q_t^\top K_{\text{tok},s}) V_{\text{tok},s}.$$

CGA instead computes

$$\text{CGA}(X)_t = \sum_{j \in E_t} \text{softmax}_j(q_t^\top c_{tj}) m_{tj},$$

where c_{tj}, m_{tj} are produced by the concept-bank update over the prefix. Attention uses one memory entry per token event; CGA uses a bounded set of concept slots that aggregate token evidence. This is why CGA can be viewed as a memory-basis replacement for attention: the read remains attention, but the state being attended to is concept-indexed rather than token-indexed.

Learning and backpropagation boundary. CGA trains its slow parameters end-to-end: embeddings, read/write projections, output projections, retention parameters, and the language-model head receive gradient from the task loss. The runtime concept update itself is not backpropagated through as a recurrent state trajectory. Routing decisions, key sums, counts, and slot priors are detached fast state. The gated value content remains differentiable so that future concept reads can train the value projection, write gate, and retention decay. Thus CGA is neither a backprop-free language model nor a test-time optimizer over model parameters. It is an attention-compatible state layer whose fast memory update is a forward-time associative learning operation while slow neural parameters are trained by ordinary backpropagation.

Algorithm 1 CONCEPTGRAPHATTENTIONFORWARD

Input: hidden states $x_{1:T}$, concept count C, block size B.
Initialize fixed concept bank with random priors a_j, edge predicates e_j, counts n_0, and zero values.
for blocks $[s, e)$ **do**
    Project block queries, write keys, values, gates, and retention rates.
    Read edge-eligible concept slots with dense attention, falling back to all slots when needed.
    Emit block outputs through the output projection.
    Route each valid write to the most similar concept slot.
    Decay old sufficient statistics and scatter-add gated count, key, and value updates.
end for
Return output projection of concept readouts.

3 Experiments

The empirical claim is intentionally narrow. We evaluate CGA as a causal sequence mixer: a layer with the external contract of attention, but whose memory is a bounded concept bank rather than all previous tokens. The main experiment is multi-query associative recall (MQAR), because MQAR directly measures whether a mixer can write key-value information and retrieve it later. Language modeling is the scaling target, but the first publication-level result should establish controlled recall and state scaling.

3.1 Experiment 1: Multi-Query Associative Recall

MQAR inputs contain key-value pairs followed by queries; the model must output the corresponding value [Arora et al., 2024]. This is the cleanest test for CGA because the task separates two questions: whether the model can store associations in a runtime memory, and whether later queries can retrieve them. For token attention, the memory is the raw prefix. For CGA, the memory is a fixed concept bank whose slot contents are updated from the prefix.

We run CGA inside the existing PoST/Zoology MQAR framework. The comparison is state-equalized against existing efficient sequence mixers. We use the same grid as the PoST/Mamba benchmark runs:

$$(d_{\text{model}}, H) \in \{(512, 4), (512, 8), (256, 4)\}, \qquad d_{\text{state}} = d_{\text{model}}/(2H).$$

These correspond to target runtime-state budgets $d_{\text{model}}^2/H \in \{64\text{K}, 32\text{K}, 16\text{K}\}$. For MQAR, a concept slot is intended to hold an association, so the concept capacity is tied to the largest training key-value count rather than to the sequence length. The curriculum reaches 128 key-value pairs, so the main grid sets $C = 128$ for every CGA configuration. All slots are active from the first block.

For each grid point we then choose a per-head concept value bottleneck d_v^c so that the persistent CGA state

$$S_{\text{CGA}} = C\,H(d_a + d_v^c + 1)$$

does not exceed d_{model}^2/H. The value bottleneck is rounded down to an eight-dimensional alignment. This gives $(C, d_v^c) = (128, 104)$ for $(512, 4)$, $(128, 24)$ for $(512, 8)$, and $(128, 16)$ for $(256, 4)$, all within the same target state budgets. Each grid point is run with the same three-point learning-rate sweep

$$\{10^{-3.5}, 10^{-3}, 10^{-2.5}\}.$$

The implementation uses dense concept reads, untied read/write address projections, a learned read logit scale initialized at the standard dot-product attention scale, a learned write gate with sparsity coefficient 10^{-4}, a fixed slot prior count $n_0 = 1$, and one fixed edge predicate per concept slot derived from the random slot priors. The main run uses the PoST parameterization for the retention spectrum, with $\text{dt} \in [10^{-6}, 3 \cdot 10^{-5}]$, train length 512, and position-adaptive scaling enabled. Training uses sequence length 512 with the same curriculum over key-value count as the PoST baselines. Evaluation reports the existing persist and capacity slices at lengths $512, 1024, 2048, 4096$.

Baselines. The comparison class is the standard MQAR suite already present in the lab framework:

- **Transformer**: token-level causal self-attention.

- **Mamba/RWKV/GDN/GLA/RetNet**: recurrent or linear-state efficient mixers.
- **TTT-style layers**: expressive state updated by test-time learning style mechanisms.
- **CGA**: attention over a fixed runtime concept bank.

What the experiment tests. The fair-state run asks whether replacing token memory by concept memory is competitive at the same state budget. The important diagnostic slices are:

$$(\text{test length},\ \text{key-value count},\ \text{persist/capacity}).$$

A positive result is not merely low training loss. CGA should maintain recall when the query distance grows, expose a compact runtime state, and report latency so that improvements are not hidden inside unbounded memory. We additionally log the mean write gate, valid-token write gate, write mass per token, assignment rate, slot-usage entropy, slot-load coefficient of variation, edge count, read scale, mean retention, and mean decay rate. These diagnostics are part of the main run because the central failure mode for CGA on MQAR is not only optimization loss, but whether useful key-value writes can be selected without collapsing all evidence into a small number of concept slots and whether the retention time constants are compatible with the sequence length.

The implementation artifact is `../PoST_dev/zoology/zoology/experiments/cga_mqar.py`. The registered mixer is `zoology.mixers.cga.ConceptGraphAttention`.

3.2 Experiment 2: CGA-LM

The language-model experiment asks whether the same concept-memory read/write operation can serve as a causal mixer beyond synthetic recall. The matched comparison is:

- **Tiny Transformer**: standard causal self-attention.
- **TinyCGA-LM**: the attention sublayer is replaced by CGA.

Both models should use the same tokenizer, depth, width, optimizer, training tokens, and evaluation split. The reported metrics are validation negative log likelihood, perplexity, prefill latency, decode latency, parameter count, slot-usage diagnostics, write-gate statistics, retention statistics, and runtime state size.

The language-model result is not meant to be a large-scale SOTA claim in this draft. Its role is to show that CGA is not an MQAR-only mechanism: the same layer that performs associative recall can be trained as an attention-like decoder mixer. For larger runs, CGA can be introduced into the existing PoST training stack as an attention replacement with the same external block interface.

3.3 Removed Experiments

Earlier drafts included CIFAR test-time adaptation, LoRA-style adapters, entropy minimization, readout-only language-model memory, and synthetic diagnostics. Those experiments are removed from the main claim. They test different questions and obscure the central message:

CGA replaces token attention memory with fixed runtime concept memory.

MQAR tests controlled concept-memory recall. CGA-LM tests whether the same operation can be trained as a causal language-model mixer.

4 Related Work

Efficient sequence mixers and expressive state. Transformers use content-addressed attention over previous tokens [Vaswani et al., 2017]. Mamba, RWKV, GDN, GLA, RetNet, and related models replace attention with efficient recurrent or linear state updates [Gu and Dao, 2023, Peng et al., 2023, Yang et al., 2025, 2024, Sun et al., 2023]. TTT-style models instead make the hidden state expressive by running a learned update rule at test time [Sun et al., 2025]. Our experiments place CGA in this comparison class: CGA is an attention-like causal mixer whose persistent state is a bounded concept memory rather than a token table.

Test-time learning layers. TTT layers frame sequence modeling as test-time learning: the hidden state is a small model, and the state update is a self-supervised training step on the current sequence [Sun et al., 2020, 2025]. CGA adopts the same high-level goal, namely to let inference-time computation learn from the observed prefix, but changes the learning object. The hidden state in CGA is not a linear model or MLP whose weights are updated; it is a concept-indexed sufficient-statistic memory. The update is therefore forward-only: route a write, decay old statistics, and scatter-add gated key/value/count statistics. This makes CGA complementary to TTT. TTT explores expressive hidden states as trainable inner models; CGA explores expressive hidden states as writable concept memories with attention-style readout.

Attention variants. Sparse attention, sliding-window attention, grouped-query attention, and compressed-memory attention mainly alter which token keys and values are read or how the token table is stored. CGA is orthogonal: it changes the basis being attended to. The read still has the same softmax form, so optimized attention kernels and attention variants can in principle be adapted to concept slots instead of token positions.

Associative and external memory. Cache language models, kNN-LMs, retrieval-augmented systems, fast weights, online ridge or RLS memories, Memory Networks, Neural Turing Machines, and Differentiable Neural Computers all use forms of stateful retrieval or local memory repair [Grave et al., 2017, Khandelwal et al., 2020, Lewis et al., 2020, Ba et al., 2016, Schlag et al., 2021, Crammer et al., 2006, Weston et al., 2015, Graves et al., 2014, 2016]. CGA shares the associative-memory goal but keeps the decoder-block interface of attention: each token reads from concept slots and writes back through a gated causal update.

Routing and capacity. Mixture-of-experts, Switch Transformers, and Product-Key Memory route tokens or queries to trained capacity [Shazeer et al., 2017, Fedus et al., 2022, Lample et al., 2019]. CGA also routes writes, but the routed capacity is runtime state rather than a bank of trained expert parameters. The fixed concept count C makes this capacity explicit and state-equalizable against recurrent mixers.

5 Limitations and Conclusion

- CGA's dense read is stable at small concept budgets, but large concept banks will need fused kernels and eventually sparse concept retrieval.
- The current retention decay is trainable and PoST-parameterized by default, but its best time-scale allocation for large language modeling remains empirical.
- Fixed-bank CGA removes online slot-allocation complexity, but it makes the concept count C a real capacity hyperparameter.
- CGA's test-time learning is prefix-local in the current formulation: the concept bank is reset per sequence unless a separate persistent-memory protocol is added.
- The experiments are sequence-modeling tests, not a complete large-scale language-model SOTA claim.

The empirical claim is deliberately narrow: CGA is evaluated as an attention-like causal sequence mixer on MQAR and small language modeling. MQAR should show whether the concept bank can write and retrieve key-value information under matched state budgets. CGA-LM should show whether the same mechanism can replace token attention in a trainable decoder. If CGA loses to optimized recurrent mixers, that is a real systems or modeling gap rather than a wording problem. The method still needs stronger fused kernels, larger-scale LM runs, retention-time-scale studies, and distributed concept-bank implementations before it can be treated as a large-model systems result.

The main point is the memory-basis replacement. Attention reads from token-indexed memory. CGA reads from concept-indexed memory whose contents are updated by a causal forward-time rule during inference. This gives a clean attention-compatible path for testing whether concept-level test-time learning can compete with token memory, recurrent state, and TTT-style hidden-model updates under matched state budgets.

References

Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *International Conference on Learning Representations*, 2024.

Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Edouard Grave, Moustapha M. Cisse, and Armand Joulin. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014.

Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626): 471–476, 2016. doi: 10.1038/nature20101.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, et al. RWKV: Reinventing RNNs for the transformer era. *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366, 2021.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *International Conference on Machine Learning*, 2024.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

Appendix

A CGA-LM: Replacing Token Attention with Fixed Concept Attention

For language modeling, the world state is a prefix and the consequence is the next token. CGA-LM replaces the causal attention sublayer inside a decoder block with Concept Graph Attention:

$$\mathrm{CGABlock}(X) = X + \mathrm{CGA}(\mathrm{Norm}(X)).$$

The block has the same external shape contract as attention, $[B, L, d_{\mathrm{model}}] \mapsto [B, L, d_{\mathrm{model}}]$. The surrounding MLP, residual stream, layer normalization, embeddings, and language-model head are otherwise standard decoder components.

A.1 Causal concept attention

At layer ℓ, token t produces read query, write key, write value, write gate, and retention-decay projections:

$$q_t = Q_\ell x_t, \qquad k_t = K_\ell x_t, \qquad v_t = V_\ell x_t, \qquad g_t = \sigma(w_g^\top x_t + b_g).$$

The layer-local CGA state is a fixed concept bank

$$\mathcal{M}_t^\ell = \{(a_j, s_{jh}^k, s_{jh}^v, n_{jh})_{h=1}^H\}_{j=1}^C.$$

The fixed slot priors also define static edge predicates that restrict each read to an eligible subset of concept slots, with fallback to the full bank when no slot is eligible. The read is causal: token t reads only concept contents produced by earlier writes. For eligible concept slots E_t^ℓ,

$$\alpha_{thj} = \mathrm{softmax}_{j \in E_t^\ell}\left(\gamma_\ell q_{th}^\top c_{jh}\right), \qquad z_{th} = \sum_{j \in E_t^\ell} \alpha_{thj} m_{jh}, \qquad y_t = O_\ell[z_{t1}; \ldots; z_{tH}].$$

Multi-head CGA uses independent address and value subspaces per head and concatenates the head outputs before the output projection, exactly as multi-head attention does.

A.2 Read before write

The update order is the same during training and autoregressive inference:

$$\text{read from } \mathcal{M}_t^\ell \to \text{emit } y_t \to \text{decay and write } (k_t, v_t, g_t) \to \text{update } \mathcal{M}_{t+1}^\ell.$$

This prevents a token from reading its own write. The inference-time update is the test-time learning mechanism in CGA-LM: the model weights are fixed, but the concept bank changes after every causal block and future tokens condition on that updated bank. Both MQAR and language-model variants use a learned write gate. For general text, the update includes a trainable retention factor $\rho_h = \exp(-\sum_t \lambda_{th})$ over old concept statistics, with the default PoST-style ordered decay parameterization and an optional free per-head parameterization for ablation.

A.3 Training boundary

Slow neural parameters are trained by ordinary next-token backpropagation through the CGA read path. The concept-bank routing state is detached:

$$\mathcal{M}_{t+1}^\ell = \mathrm{CGAWrite}\left(\mathrm{stopgrad}(\mathcal{M}_t^\ell), \mathrm{stopgrad}(k_t), v_t, g_t, \rho_t\right).$$

Thus CGA-LM is not a backprop-free language model. It is an attention replacement whose fast state is updated by a forward-time concept-memory rule while the slow projections are trained end-to-end. The gated value content and retention factors remain differentiable through future reads; routing, key statistics, counts, and slot priors are detached.

Algorithm 2 CGALAYERFORWARD

Input: hidden states $x_{1:L}$, fixed concept bank $\mathcal{M}$
for causal blocks **do**
    project block queries, write keys, and values
    read edge-eligible concept slots with continuous attention weights
    emit block outputs through the output projection
    form write pairs, write gates, and retention factors
    route writes to concept slots
    decay old concept statistics, scatter-add new statistics, and commit bank updates
end for
return contextualized hidden states

A.4 Scaling path

The reference CGA implementation uses dense concept reads. This is appropriate for state-equalized MQAR where the concept budget is small. For larger concept banks, fused dense-read kernels, sparse concept retrieval, and sharded concept banks are systems work. The paper therefore separates the architectural claim from the kernel claim: CGA changes the memory basis of attention from tokens to concepts; optimized implementations should then choose dense or sparse concept attention depending on the concept budget.

NeurIPS Paper Checklist

This file is retained only for the conference-template branch of `main.tex`.

- **Claims.** The abstract and introduction state the paper's scope as a method proposal: CGA replaces token-indexed attention memory with a fixed bank of runtime concept slots.
- **Limitations.** Section 5 states the main limitations, including kernel optimization, retention tuning, concept-count selection, and the absence of a large-scale SOTA language-model claim.
- **Theory.** Section 2 gives the state definition, read/write equations, state-size accounting, and backpropagation boundary for fixed-bank CGA.
- **Experiments.** Section 3 centers on MQAR as the controlled recall benchmark and CGA-LM as the sequence-modeling scaling target.